

Video Transcripts

Module 13: Change Data Capture (CDC)

Video 1: Introduction (01:44)

As the saying goes, change is inevitable; change happens. When it comes to data, the same thing is true. From your single source of truth, you might have a number of downstream systems that are concerned with this change that is needed. When it comes to caches, something that is essential for speed, you need to update those caches. When it comes to data scientists, they need to work with those new changes that are coming from your single source of truth. Now think about search as well. The results that people are finding need to be updated with the new information. So, when it comes to change data capture, this is a problem that many have recognized, and efforts have gone to address it.

Now, we'll begin by writing our own change data capture approach, where we will propagate information through a number of databases. We'll start with a relational, then move to a document, then move to a key-value pair, end up with a distributed, decentralized database. We will be propagating through all of this. Now because of it, you can imagine there's quite a bit of complexity. We want to fully automate the system. We'll start off with shell commands, and we will build some scripts. We will definitely make heavy use of containers. We will be initializing databases, clearing things out, then creating an event loop that moves through all of this. So, this is a great project to dig into and have a lot of fun. It brings together your knowledge of databases, of containers. You will be writing some scripts here and bringing all of it together under a Python shell. So, go ahead and enjoy.

Video 2: Change Data Capture (CDC) Overview (04:59)

Change is inescapable. It is one of the constants in life. In fact, we as humans constantly track that change. There's even an expression right fear of missing out. Now, when it comes to data, the same thing is true. Once you create a database, there are a number of parties that are interested in changes to that database. Now, when it comes to data capture, as you might imagine this is not new. It has been around for decades and perhaps one of the most tangible

examples is that of vanity phone numbers. Most countries around the world have pretty numbers that they often sell to businesses that then map to regular numbers. Now, every single time there is a change to one of these, the hundreds of database servers around the phone network need to be updated to reflect that change. Something similar happens with the domain name server system, the mapping of pretty domain names like mit.edu to IP numbers around the world. Every single time there's a change to one of those, that update needs to happen, in this case globally.

Now, one of the twists when it comes to change data capture is that we're looking at real time data or as close as we can get to it. You might be wondering, well, what's the big deal? We have a database and we have an application. Why can't we do that? Well, as it turns out, when that application scales, you get more needs. You might no longer be able to meet the demand and you need to add additional performance. Which means you will add a cache. Same thing happens to being able to find information within your application. You may need a search index and an additional one would be a data warehouse. These needs grow and you ultimately end up with a system of record, a source of truth, and then the right data. Once you do that, you need to synchronize that information. And when it comes to the ways in which you can do that, there are a number of them. I'll list here four of them, dual rights, looking at it from the network, looking at it from the database, and then periodic queries.

Now, let's look at each one of these in turn. As you can see here, this is dual writes, and this might be your first reaction. Let's just update our source code and let's write to the number of places that need this data. Well, it turns out that's a pretty bad idea. For one, you have to update your source code. This might not be something you want to do. The other is that this turns out to be non-transferable. That is the code that you're writing is specific to that application. And in general, there are a number of problems that emerge because once again, this might be a natural thing to do at first that simply cause all sorts of problems. And data engineers like to say friends don't let friends do dual writes.

Now, another one that you might think of is network level sniffing. This is similar to the screen scraping that Google does. They navigate the World Wide Web and scrape the pages and try to create a data model from it. Now, this is complex enough on its own. However, you can imagine that one of the additional things that you miss if you take this approach are the intermediate changes of state that happen inside the database. The other issue as well is that if this

application crashes, the sniffing application crashes. You miss everything that happens because there is no redundancy to it. Now, the third way you might look at this is to look at it from the database perspective. And if you are a database person, you tend to think in terms of databases. As you can see here, there are a number of issues with addressing it that way. As mentioned, a lot of these issues are emerged as your application scales. And if you're adding additional load, you know you might have to do multiple writes to multiple sources. This is going to slow down further your application.

Now, you can see additional scenarios there where you know if a transaction fails, you'll have less updates. But in general, we have a lot of the same issues when it comes to transferability that we had in dual writes. The logic that you're writing is specific to this application. Now, a fourth way, and this is a pretty good way when it comes to DIY type of approaches, is to do periodic queries and leverage timestamps in the database. If done right, this can be pretty low overhead. Now, it does have some disadvantages that you can see there. Depending on how you write it, you could address those or they might be big deals. Now, the last thing to mention when it comes to CDC is that you're looking really at three core components. You're looking at the detection of the change, you're looking at capturing that change, and you're looking at propagating the change. When you think in terms of CDC, think in terms of these three core components.

Video 3: Basics of a CDC System (02:41)

Let's build a change data capture system that is a CDC system. We're going to write or we're going to pretend that we have a very simple application that simply writes a timestamp into the database. As you can see here, it will be written into MySQL. We're then going to react by detecting the change, capturing the change, and then propagating it throughout the tiers that you see there. MySQL will propagate to MongoDB, that will be propagated to Redis, and that will be propagated to Cassandra. We will use a periodic queries approach and you can see here the flow chart. We will start off by doing a query. If we detect the change, then we will select the new data, and then we will propagate that data that is save or share. When it comes to the organization, MySQL will be the system of record, the source of truth, and then we will be repeating that the tag capture and propagate through the tiers. Now, as you can imagine, writing something like this could become quite involved. You are creating containers, you are checking on the logic. There's a lot that could be going on here when it comes to creating that

entire system to be able to track those changes.

So, because of it, we're going to write an automated system, and we're going to break it down into components to make it more granular, to be able to test, and to be able to ensure that we have, in fact, the functionality that we think we do. So, we're going to write all of this in Python, and there will be a number of components. The first one will be one through which on which through Python, we will be writing shell commands. We will be able to create containers through that as well and then delete the containers. When we're moving through a fast development cycle, we want to be able to create and tear that down. That is, clean up what we are creating while we are creating our CDC system. Just as well, some databases require you to initialize them before they can be populated, so we will do that programmatically as well. Then we will clean up the databases. In some cases, we don't want to delete the entire container. We don't want to delete the database or the table. We simply want to clear out the data. Finally, we will do all of this in an event loop. That is a timer that will be going through all of these steps, and once again we will control all of this using Python.

Video 4: Embedding Terminal Window Commands in Python (03:42)

You can think of automation programs as glue that bring together a series of tasks that normally get performed independently. We'll frequently want to reach in into the shell of our operating system, as you can see here in this case, and we want to be able to do it from Python. We'll want to be able to run commands as if we were typing them directly at the command line. So, let's take a look and see how we can do that within Python. As you can see here, I have my editor open, I have a single file open called shell.py, and I'm going to go ahead and import a couple of packages, the operating system package, the system package, and then I'm going to define a couple of input arguments. This is something you frequently see in command line invocations of a program, and in this case we're writing them ourselves. The very first one is going to list the files, as you can see there.

The second one is going to list hidden files, a very simple thing to do simply to illustrate how to write a program that runs command line instructions. So, after that, we're going to define our two functions. The very first one is going to list files, and as you can see here, this is the command to be able to list files. Then the very next one is going to do the same thing, except that is going to list hidden files. And you can see here how we pass the command onto the shell

and how we catch those results. And in this case, we're going to be printing them to the console. Then once we do that, the next step is that we're going to read those input arguments. We talked about two of them, one to list files, one to list the hidden files. And you can see here that we're going to read those arguments from the command line. The very first one would be the name of the program and the second one or the Python file, and the second one would be the argument.

Now, once we do that, you can see here that I'm going to check for what argument was passed then and if it's list, then I'm going to call listfiles. Otherwise, I'm going to request list hidden files, as you can see. So, I'm going to go ahead and save that. Now, I'm going to go ahead and open up the console and I'm going to go ahead and run that program. So, I'm going to go ahead and enter python3, the name of the program. And first, I'm not going to enter anything. As you can see there, nothing happens. Then I'm going to go ahead and enter list for list files. And as you can see there we have a single file in that directory. And so we get a listing of the files. Now, let's go ahead and create a hidden file. So, I'm going to go ahead and enter touch and I'm just going to write here hidden. Now, if I enter list again, you'll see that I only get one file because this is the just the normal listing. But if I enter hidden, then I should be able to see both and I can. So, as you can see there, this is a pretty convenient way to be able to run commands at the command line, but to do so programmatically within our Python program. So, go ahead and give it a try, see if you can replicate, explore a little bit of what you could do, and make sure you're comfortable running command line arguments through Python.

Video 5: Creating *Containers* Using Python (03:31)

Creating containers by hand, they're certainly powerful. It gives us great speed and capabilities. This is what we have used to be able to bypass the local configuration and set up that can be quite lengthy. However, when we're creating a lot of these, doing it by hand, one by one, at the command line can be slow. And so it would be much better to have a Python program that ran all of those commands against the command line on the shell for us. So, let's take a look and see how we can do that. As you can see here, I'm in my editor where I have a single file called Pythos.py and we'll start off here by adding two packages, the operating system and system package. Then we'll define an input argument, which in this case is called create. After that will come the function that we will use to create our containers. Note that it takes two parameters command in the database, and you can see here that in line 11, we're going to be creating that

container, and if that goes well, we're going to use the name of the database simply to update the command line and know that we have successfully created that container.

Then we're going to use our block there, lines 15 through 18 to be able to read that input argument. And after that we will have a conditional statement to be able to go into that block of code to be able to create the container. As you can see here in this case, I'm going to be creating a container Docker run of my SQL and the rest of it are the parameters that we have used before. So, let's go ahead and save. I've gone ahead and moved to the desktop and I have opened a terminal, and if I request the path here or show you the path, you can see there that I am in the same folder. If I list the files, you can see there that I have containers and I've also opened my Docker dashboard to be able to see the container when it gets created. So, now let's go ahead and run our program, Python3, and the name of the file is containers.py. And then we're going to give that argument that we defined.

So, let's go ahead and hit return. And as you can see there that is pulling the container as we had expected and that is going through all of the setup and configuration that is needed. And you can see there that the dashboard just updated and I have a running instance of MySQL running on port 3306. And you can see there that it's nice and green, which means it's now available and ready for me to use it. So, as you just saw, I went ahead and created the container programmatically for MySQL. It is now your turn. First, make sure you can replicate what I did, that you can create the MySQL container. Then use the command line instructions that we have used for the other databases, the Mongo database, Redis and Cassandra. And make sure you can add those instructions to that program and that you can create all of those containers in one step. It will make you feel powerful and it'll simply facilitate you being able to create all of these in one shot.

Video 6: Deleting Containers Using Python (02:14)

We can certainly delete containers by hand. We can stop the container, we can then delete it. However, if we have a long list of them, that could be quite tedious. Plus, if we're creating an automation script where we're going to be creating, doing some work and then deleting, it's quite convenient to be able to do it programmatically. So, that is precisely what we are going to be doing next. As you can see here, I have a single file called containers.py and I'm going to go ahead and bring in the operating system package and the system package. I'm then going to

define an argument called `-delete` to be able to delete my containers. And then you can see here the function that will actually carry out that work.

As you can see there, I pass in the name of the container and the very first step on line 11 is to stop that container. Then I am going to go ahead, if that result is successful, I'm going to go ahead and delete that container next. And finally, I will simply update the command line to be able to show that I have deleted that container. Now, I'm going to go ahead and read that argument, the argument of `delete`, and if I get a match, then I'm going to pass in the name in this case that I have given my SQL Server, which is some SQL. So, I'm going to go ahead and save the file, and I'm going to go ahead and show my docker dashboard as you can see there. And then I'm going to go ahead and run that program. And if it's successful, we should see that disappear.

So, let's go ahead and write containers and in this case, `delete`, `-delete`. We have gone ahead and sent that command. And as you know from experience, doing it by hand, it takes just a little bit. And that was that pause you saw, but now it's gone. So, this was done here for my SQL Server. Now, it is up to you to be able to delete all of the other ones you have created. So, in a previous exercise, you created four different databases programmatically. Now, if you still have them up, you can delete them all programmatically.

Video 7: Initializing Databases (03:51)

Some databases like Mongo and Redis are ready to go out-of-the-box. You create the container and you're ready to go. You can start writing to them. But others require more structure. Relational database management systems require you to create a database, to create a table before you can start writing to them. And because of that, you might want to have a block in your automation script to be able to initialize databases. And so that is precisely what we will do. We will do it for my SQL. And as you can see here, I have a file ready to go. I'm going to start off by importing the OS and system packages. I'm then going to define two arguments as you can see there. One for `create` to create the containers and another to `init`. And I'm noting there once again that for certain databases you need to `init`, for others it is not needed. The very first block, the very first function is going to create the container. The next block is going to do the `init`. On my SQL, you can see there that I am connecting to the container. I'm then going to create a cursor, make sure that database doesn't exist before I delete it. In this case, it wouldn't

exist, but I'm just doing defensive programming and then I go through and I create a table and then simply clean up by closing my connections and committing my changes.

Later on, we're going to go ahead and read that argument from the command line to be able to pick it up and then decide which block of code below that is the one that we want to run. The very first one is looking at the case of creating the database, and the second one is looking at init. So, let's go ahead and save. And as you can see here, I have my Docker dashboard. So, we can see when the container is created. And we're going to start off and run the first option, which is the option to create. And so we're going to go ahead and enter Python. And then the name of the file is containers. The option or the argument we're going to pass in is the one of create. We're going to go ahead and hit return. And as you can see there, we have created a container that's up and running, nice and green. And now, we can go ahead and run the init. As you can see there, we get an error. And that is because, as you can see here, I have not added the driver to be able to connect to MySQL. So, let's go ahead and add that.

I'm going to go ahead and save. Let's go ahead and run once again. And it looks like I left some code that doesn't belong in this trimmer version. So, let's go ahead and scroll down. And as you can see, yes, I did leave Cassandra there. I will leave that for you to do as an exercise. But for now, we don't need it. We don't have the code that is going to perform that init. I'm going to go ahead and save once again. And now, I'm going to go ahead and run one more. And as you can see there, that went through without any errors. The database now has that new database and that new table. So, now it's your turn. Make sure you can replicate the code that we just went through. Make sure that you can initialize my SQL and take a look and add one more init for Cassandra. This is the additional one that you saw there in my code. The other two, Monger and Redis do not need it. So, go ahead and enjoy, dig in, now it's your turn.

Video 8: Time Loops (03:20)

When you do simulation, some sort of animation, say a visualization or in our case automation, you want to move through some time interval, and that is precisely what we're going to be doing. As you can see here, I am defining a template for a time loop. You can see there that I have a function, a number of tasks, and at the end of it I have a function that is part of the language that allows you to create a timer. It says the time at which you will call the function that you provide, and then you can fire it. As you can see there, that function is calling itself. So,

we will have a loop. So, let's go ahead and write up that code. Let's go ahead and open up the timer file. We'll start off by importing time and timer. After that, we will create or we will define our tasks. These are dummy tasks, as you can see there. I'm simply printing to the console to the terminal task 1, and I'm going to make it go to sleep for a couple of seconds simply to simulate something that is taking some time to complete. Then I'm going to go ahead and add one more, which will be task 2. And then I'm going to have my function that is like the template we just discussed that has a number of tasks, and then that timer that allows us to call it ourselves. So, we inside of that function, we will execute task one or perform task one, then two.

Then we will write a timestamp to the terminal. And then we will set our timer to execute in five seconds, call itself and start. So, let's go ahead and run this. So, let's go ahead and run that file... Python 3... timer. And as you can see, nothing happened there. Let me check and see that I saved. Yes, and I can see the problem. I did not invoke the time loop, so it runs through everything, but it doesn't do anything. So, let's go ahead and save, run once again. As you can see there, immediately we start to execute task one, then task 2. Notice that there is a delay performing each one of them. Then we get our timestamp and then we start over task 1, task 2. Then we execute our next timer that calls itself and note there the time difference. As you can see there we have 54 seconds, then we are at zero-four seconds, then at 14 seconds. And the reason for that is that if you add up the seconds between the pauses there, you can see there that we have five here on line 16, then we have three on line 10 and then two in line six. So, all of them added up together are 10 seconds. So, if we look at the time stamps there, they are 10 seconds apart. So, as you can see, this this will run forever. It is a quick and dirty way to do a time loop, and it'll work fine for our objective, which is to automate a number of tasks.

Video 9: CDC – Creating a Database Using Python and MySQL (04:13)

At this point, you have all the knowledge that is needed to be able to create a full change data capture application. The full beast if you would know how to create shell commands, create containers, delete containers, initialize databases, clear out databases, and create time loops. As you can see there, we can do all of this within Python. So, at this point you could go ahead and get started on your own, but it can be a bit overwhelming to tackle the full breadth of this application. So, I'm going to go ahead and get you started, and I will start off here with MySQL and Mongo and then leave it up to you to add Redis and Cassandra. It's always a good idea to start small and then build up. We're going to be using the following architecture. As you can see

here, I have a number of files and I've given you also the names that I'm using for these files. I'll start off here from the left and as you can see there, there is going to be a file that is dedicated to containers. Through it, we will be able to create the containers for the databases, delete them, and initialize the databases that require it within them. We're also going to have one called scheduler, and you can see that there in the center of the page. And that file is going to be importing the files that are specific to each of the databases. You can see there that I have one for my SQL, one for Mongo, and so on. The Gray dot ones are the ones that are for you.

Those are the ones that you will write. Now, once we pull those in, as you can see there from the icons above, we will be able to create, read, and update information. We will also add some more logic to be able to clear out the data from the databases. So, simply to emphasize, as you can see here, the containers file, containers will be able to create the lead, initialize the scheduler file will be importing those database specific files, then do a time loop and leverage those functions. So, with that in mind, let's go ahead and move to the code. As you can see here, I have files corresponding to the ones that we just saw on the diagram, and I'm going to start off here with MySQL. This is a database server that we have used quite a bit. So, you're quite familiar with how to connect to a database, how to input information, how to read information. So, as you can see here, I start off by adding, importing the driver and then I have a couple of supporting or a few supporting packages there. Now, the next thing that I do is that I make my connection to the database, create a cursor and then I write within this function.

Note here, that I am creating a unique identifier and then a timestamp which gets pushed into the database of the database called Pluto. You can see that here on line 9 and into the table called posts. Now, it's worth noting that this is what begins the entire cycle of data capture. This is what will propagate through the rest of the databases. But this is the place where it all starts. Now, below that I have a number of additional functions. You can see there that I have the read function. This one will read the last five entries that have been made into that database, and you can see there that that gets packaged into an array, which is then returned to the calling function. I also have a method, a function below that that is simply going to clear out the information in the database. This is a good idea to do when you're working in a pretty high iteration; where you want to be able to make big changes, remove them, and then start over. Now, the last part of that is simply handling the case where we exit and that it closes the connections.

Video 10: Connecting and Writing to a Database Using MongoDB (10:37)

So, now let's go ahead and take a look at the file for Mongo. And as you can see there that is pretty similar to the one that we just looked at for MySQL. However, it is simpler because Mongo requires less setup and configuration. We make the connection to the database, we get a handle on the database app, and configuration of the database in the document world. Same goes for the table. And then we write into that table. Notice that in this case we're passing in that new information. It takes a parameter here, and that parameter then is iterated through, and then we push as you can see here on the update.

We also have an additional function below it to be able to read the data to confirm that those updates, those writes, are in fact taking place. And then just like in MySQL, I have a last function there to be able to clear out the data in the database, on the MongoDB database. So, that was MySQL and MongoDB specific files. Now we're going to take a look at the scheduler. This is the file that holds the time loop. And as you can see there, this is the file, the MySQL DB and MongoDB files. We will use these here to be able to access those functions that we just overviewed. There's some additional things that we have here right below. Note that we have, and these are in addition to the time loop that starts here on line 23. We have a function to be able to clear out all of the data that we have in both of those databases.

And as you can see there that simply calls the functions that are part of MySQL and of Mongo. Below it, we have the deployment that we pass in at the command line and if we match that argument, you can see there that it simply clears out the data that we have in those databases. Now, here is the mean time loop, and we are going to overview here the functions. The very first one is one that simply writes to the console the data, and this is a good way to keep track of what we're reading and also note that it sleeps for five seconds. We do this simply to be able to see the data that is being written to the console. Below it, we have MySQL write. This is the one that leverages that function within the MySQL file and it does it write. This is what starts everything when it comes to being able to track those changes.

The next one is Mongo, and you can see that Mongo starts off by reading from MySQL. Once it does, it writes the status using the status function to the terminal, and then it writes into the Mongo database. Below it, we have a function to verify. It reads from Mongo and then writes to the terminal once more. And below that at the very end of the file we have the time loop. Time loop is pretty simple. It starts off by doing a time stamp to the terminal. Then it calls MySQL,

followed by Mongo, followed by the verification function. Then it starts a timer which simply calls itself the function that it's in. And so, we get a time loop. And on line 55 we simply start that loop running.

Now, the last file to overview is the containers file. And as you might recall, this is the one where we delete containers, create containers, and initialize the databases that require it. So, as you can see there on my note on line 10, this is really only for MySQL, MongoDB does not need it. So, the very first block is the block to delete containers. You can see there that we are running the commands and writing acknowledgments to the terminal. The second block is to create containers. The third block is the one that initializes the database, and that runs from line 27 all the way to line 57, actually no, all the way to line 62. The block below that is the one that is reading the argument, and the three blocks, conditional blocks that follow it are the ones that are matching the kind of argument that was passed in. So, the very first one deletes containers. The second one creates containers, and the very last one is the one that initializes the database. So, that was quite a bit.

With that in mind, let's go ahead and move to the terminal and run this code. Let's start off here by listing our files, and then we're going to go ahead and run the containers file to be able to create our databases. And as you can see there, I have just opened the dashboard for Docker, so we can see when those containers are created. I'm going to go ahead and run the containers file and I'm going to use the argument for create. So, I'm going to go ahead and enter 'return'. And as you can see there, it starts off by pulling the images. I have cleared out my machine to make sure that it was clean. So, none of those images for those databases exist. And you can see there that one of the first one or the first one that was created there was MySQL, and now that is being followed by the Mongo image being pulled.

You can see there the process and the feedback that we get on the console. So, at this point, as you can see there, we have created both of those databases, and if we wanted to, we could go ahead and delete them. So, let's go ahead and delete, and this will take a bit. There's quite a bit there to remove. You can see how the dashboard gets updated, and then we can go ahead and create it. So, as you can see there, this is pretty convenient. This is very fast. It's much easier than doing it by hand. And so, this is something that you can see there the advantage of.

So, let's go ahead and do the last thing, which is initialize that database. We want to be able to initialize MySQL DB. And I'm going to go ahead and run that command. And now that has been

initialized. There has been a new database that has been added, the Pluto database. And inside of it, a table has been created, the table for posts. So, at this point, let's go ahead and clear the screen. And we're now going to run our scheduler. So, this is our main loop. This will start off populating MySQL. Those changes will be detected and then pushed into MongoDB, and we should see confirmations on the console. So, let's go ahead and run that. So, this would be Python 3 scheduler. And as you can see there, immediately we get the time stamp to the console.

You can see there that that data now has been copied into or has been updated. We have detected the change in data and, we have propagated it onto MongoDB. And you can see there the time stamps that we're getting now in the second loop or in the second time through that loop. You can see there that we have now two rows in MySQL, and you also have two rows within Mongo. And you can see there that the time stamps are being put into the terminal as we expect. Now, this is moving a little bit slow, and I'll show you how to make it faster. So, I'm going to go ahead and interrupt it here. As you can see there it complains, but that's ok. As you can see here, I am in the scheduler file and I'm going to go ahead and make the time loop go faster every five seconds, and the status pause, I'm going to make two seconds.

So, now I'm going to go ahead and restart. And as you can see there, I have just removed the dashboard because we don't need it anymore of Docker containers. I'm going to go ahead and clear here my terminal and then I'm going to go ahead and run the scheduler once more. As you can see there, we are reading the data in MySQL, then the data in Mongo. Then we should get the time stamp. As you can see there, we got it. And you can see there that that is now moving faster. Note that as time goes by, the very first row that is read is different. And you can see there that the later timestamps are being pushed out of those five that are being selected. So, go ahead and take a look at the functioning, running program here. And as you start to get some results, make sure that you can match the logic that is being captured, and the way in which information is being propagated in this program.

Now, there's one more thing here to demonstrate. I'm going to go ahead and stop the program. And there was an option, if you remember, to be able to clear the data within both of those databases. I'm going to go ahead and run it. And as you can see there, we have cleared all of the information. If I was to run this scheduler again, you can see there that we start with empty databases. Again, a convenient thing to do when you're testing logic and you want to start over frequently. You'll want to write yourself that function to be able to clear out the data and start

clean once again. Now, you can also delete the containers, but that takes a little bit longer, as you can see. Delete the container, delete. We're moving entire databases, and so it is more convenient often when it comes to databases to simply clear out the tables, clear out the databases, because that is a much quicker operation. So, now it is your turn.

Video 11: CDC - Connecting and Writing to a Database Using Redis and Cassandra (00:53)

It is your turn to do the full beast. As you can see there, we simply looked at two of those databases, MySQL and MongoDB. You'll want to follow the same approach. You can use the same architecture, simply add more files, one for Redis, one for Cassandra. Simply add those additional functions in the time loop, and you have there, very good hooks, a very good structure that should expand easily to that extra functionality. There is a great deal here to dig into. This is a great one to push on your knowledge of databases, on your understanding of time loops, on your understanding of programmatically creating databases. So, this one will teach you a lot.

This is a good one to spend a good amount of time on.

